

Bayesian Neural Networks and Monte Carlo Dropout

ELG 5218 - Uncertainty Evaluation in Engineering Measurements and Machine Learning

Miodrag Bolic

University of Ottawa

February 25, 2026

Roadmap

- 1 BNN Training Loop
- 2 MCMC for BNNs
- 3 Variational Inference: Bayes by Backprop
- 4 MC Dropout: Training and Inference
- 5 Why VI when MC Dropout is so Simple?
- 6 Diagnostics
- 7 Summary

What This Lecture Covers (Implementation Focus)

How methods are implemented:

- 1 BNN training loop and heteroscedastic loss
- 2 MCMC for BNNs: SGLD, HMC — practical algorithms
- 3 Variational inference: Bayes-by-Backprop in detail
- 4 MC Dropout: exact training and inference procedure
- 5 **Why VI when MC Dropout is so simple?** — decision guide

Key references

Gal & Ghahramani (2016)
Blundell et al. (2015)
Kendall & Gal (2017)
Welling & Teh (2011) — SGLD

From Point Estimates to Weight Distributions

Standard (deterministic) NN

- 1 Choose loss $\mathcal{L}(w)$ (NLL, MSE, ...)
- 2 Minimise: $\hat{w} = \arg \min_w \mathcal{L}(w)$
- 3 Predict: $\hat{y} = f_{\hat{w}}(x^*)$

$\hat{w}$ is a **single point** in parameter space.

Bayesian neural network (BNN)

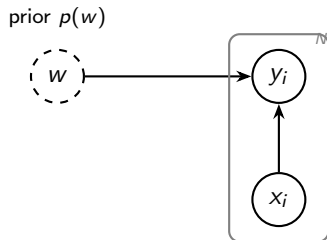
- 1 Specify prior: $p(w)$
- 2 Observe data: $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$
- 3 Infer posterior: $p(w | \mathcal{D}) \propto p(\mathcal{D}) p(w)$
- 4 Predict: $p(y^* | x^*, \mathcal{D}) = \int p(y^* | x^*, w) p(w | \mathcal{D}) dw$

$p(w | \mathcal{D})$ is a **distribution** over parameters.

Physical intuition for an engineer

$p(w | \mathcal{D})$ encodes **which model configurations are consistent with the data**. Regions of input space with few training samples $\Rightarrow$ many plausible $w \Rightarrow$ wide predictive distribution.

Graphical Model View of a BNN



Bayes' theorem for BNNs

$$\underbrace{p(w | \mathcal{D})}_{\text{posterior}} \propto \underbrace{p(\mathcal{D})}_{\text{likelihood}} \cdot \underbrace{p(w)}_{\text{prior}}$$

Likelihood models

Regression: $y_i | x_i, w \sim \mathcal{N}(f_w(x_i), \sigma^2)$

Classification: $y_i | x_i, w \sim \text{Cat}(\text{softmax}(f_w(x_i)))$

Common priors

$$p(w) = \prod_j \mathcal{N}(w_j; 0, \tau^2)$$

(i.i.d. zero-mean Gaussian, equivalent to L2 regularization at MAP)

Posterior Predictive = Bayesian Model Averaging

Posterior predictive distribution

$$p(y^* | x^*, \mathcal{D}) = \int p(y^* | x^*, w) p(w | \mathcal{D}) dw \approx \frac{1}{S} \sum_{s=1}^S p(y^* | x^*, w^{(s)}), \quad w^{(s)} \sim p(w | \mathcal{D})$$

Intuition: each $w^{(s)}$ defines a different network function $f_{w^{(s)}}$. The BNN *averages over many plausible networks*.

Why this differs from an ensemble

In a BNN the samples $w^{(s)}$ come from the *posterior* (data-informed), not from re-training with different random seeds.

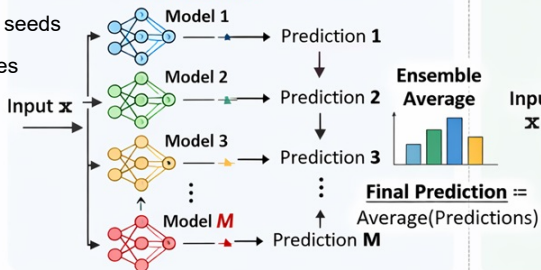
Deep Ensembles vs. BNNs

Differences Between Deep Ensembles and BNN

Deep Ensembles

Multiple **Independent** Neural Networks

different random seeds
different weights
different outcomes



Characteristics:



Train Multiple Models **Independently**



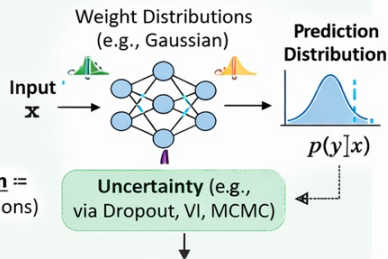
High Computational Cost



No Uncertainty Estimate (only via variance)

Bayesian Neural Networks (BNN)

Single Network with **Uncertainty**



Characteristics:



Single Model



Computationally Efficient

(inference only)



Provides **Uncertainty Estimates**



Better for Small Data

Why the Posterior is Intractable

The posterior requires normalising the likelihood:

$$p(w \mid \mathcal{D}) = \frac{p(\mathcal{D}) p(w)}{p(\mathcal{D})}, \quad p(\mathcal{D}) = \int p(\mathcal{D}) p(w) dw.$$

Practical difficulties

- $w \in \mathbb{R}^d$ for $d = \text{thousands--billions}$ in modern nets
- The integral $p(\mathcal{D})$ is computationally intractable (high-dimensional, non-Gaussian)
- The posterior is **multi-modal**: many local minima, complex geometry

Three families of approximations

- 1 **MCMC** (HMC/NUTS, SGLD): stochastically sample $w^{(s)} \sim p(w \mid \mathcal{D})$ — asymptotically exact, expensive
- 2 **Variational inference**: optimise a tractable $q_\phi(w) \approx p(w \mid \mathcal{D})$ — fast, biased
- 3 **MC Dropout**: use dropout masks as an implicit variational distribution — cheapest, good baseline

BNN Training: The Heteroscedastic Loss

Standard regression outputs a single value. A BNN regression head outputs *both* mean and log-variance:

output: $(\hat{\mu}_w(x), \log \hat{\sigma}_w^2(x))$

Heteroscedastic NLL loss

$$\mathcal{L}_{\text{het}} = \frac{1}{N} \sum_{i=1}^N \left[\frac{(y_i - \hat{\mu}_w(x_i))^2}{2 e^{s_i}} + \frac{s_i}{2} \right]$$

where $s_i = \log \hat{\sigma}_w^2(x_i)$.

Why not MSE? MSE ignores aleatoric noise. Large-noise samples should contribute less — the heteroscedastic loss provides this automatic down-weighting.

PyTorch implementation

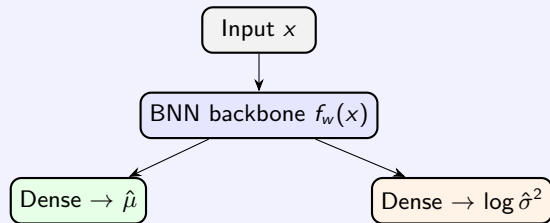
```
def hetero_nll(y, mu, log_var):  
    # y, mu, log_var: (B,)  
    prec = torch.exp(-log_var)  
    return 0.5 * (  
        prec * (y - mu)**2  
        + log_var  
    ).mean()
```

Numerical tip

Predict $\log \hat{\sigma}^2$, not $\hat{\sigma}^2$, to guarantee positivity and avoid gradient explosion.

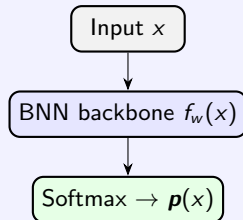
BNN Architecture: Head Design for Uncertainty

Regression: dual-head design



Loss = heteroscedastic NLL.

Classification: softmax head



Loss = cross-entropy.
Uncertainty from MC-averaged softmax, not from the architecture.

Where dropout goes

Dropout is placed *inside* the network (after hidden layers). The heads themselves typically do *not* have dropout so that mean and variance outputs are not further masked.

Stochastic Gradient Langevin Dynamics (SGLD)

HMC requires full-batch gradients and is too slow for large nets. **SGLD** injects Gaussian noise into SGD updates to obtain asymptotically correct posterior samples.

Too Slow for Metropolis-Hastings

Hamiltonian MC? NUTS?

SGLD update rule

$$w_{t+1} = w_t - \frac{\epsilon_t}{2} \nabla_w \tilde{U}(w_t) + \eta_t, \quad \eta_t \sim \mathcal{N}(0, \epsilon_t I)$$

Gradient Ascent

where $\tilde{U}(w) = -\frac{N}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log p(y_i | x_i, w) - \log p(w)$ is the stochastic potential (negative log-posterior, mini-batch scaled).

Full SGLD algorithm

```
1: Initialise  $w_0$ ; set step schedule  $\{\epsilon_t\}$ 
2: for  $t = 1, 2, \dots$  do
3:   Sample mini-batch  $\mathcal{B} \subset \mathcal{D}$ 
4:   Compute stochastic gradient  $g_t = \nabla_w \tilde{U}(w_t)$ 
5:   Sample  $\eta_t \sim \mathcal{N}(0, \epsilon_t I)$ 
6:    $w_{t+1} \leftarrow w_t - \frac{\epsilon_t}{2} g_t + \eta_t$ 
7:   if  $t > t_{\text{burn-in}}$  and  $t \bmod K = 0$  then
8:     Save  $w_t$  as a posterior sample
9:   end if
10: end for
```

Run this network multiple times to learn

Intuition

SGD = gradient descent.
SGLD = SGD + noise injection.

The noise prevents the chain from collapsing to a mode.

The step size schedule $\epsilon_t \rightarrow 0$ ensures the chain converges to $p(w|\mathcal{D})$ rather than oscillating.

Practical notes

- Burn-in: discard first $t_{\text{burn-in}}$ samples (chain not yet mixed)
- Thinning: save every K -th sample to reduce correlation
- Step size matters: too large $\Rightarrow$ inaccurate; too small $\Rightarrow$ slow mixing
- Cyclical SGLD (Zhang et al. 2020) improves sample diversity

SGLD: Posterior Predictive and Inference

After collecting S posterior samples $\{w^{(1)}, \dots, w^{(S)}\}$:

Monte Carlo predictive

$$p(y^*|x^*, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S p(y^*|x^*, w^{(s)})$$

$$\hat{\mu} = \frac{1}{S} \sum_s f_{w^{(s)}}(x^*), \quad \hat{\sigma}_{\text{ep}}^2 = \frac{1}{S} \sum_s (f_{w^{(s)}}(x^*) - \hat{\mu})^2$$

$$\hat{\sigma}_{\text{al}}^2(x^*) = \frac{1}{S} \sum_{s=1}^S \sigma_{w^{(s)}}^2(x^*)$$

- **Epistemic** = variance of the means across weight samples: "If I draw different weights from the posterior, how much do the **predicted means** disagree?"
- **Aleatoric** = mean of the **per-sample noise** variances: "On average across weight samples, how much irreducible observation noise does the model predict at this point?"

When to use SGLD over MC Dropout

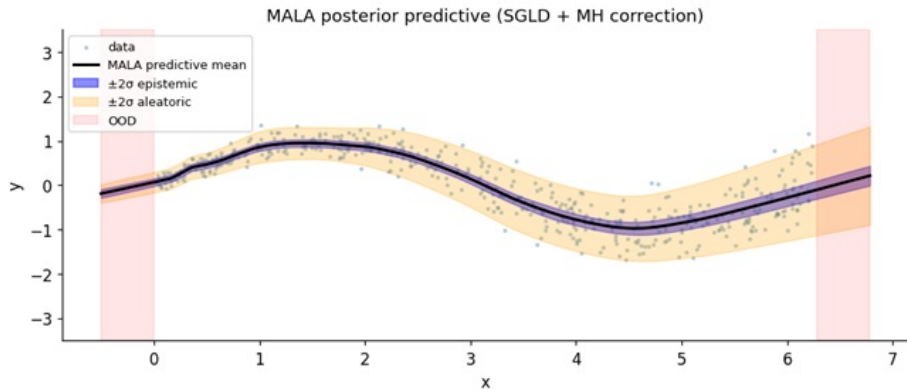
Use SGLD when:

- The model has *no* dropout layers
- You need a benchmark posterior (e.g. for validating VI)
- Dataset is small-to-medium (fit in GPU memory or fast iteration)
- You need asymptotically correct samples

Drawback:

Storage of S full model checkpoints ($\sim S \times$ model size).

SGLD Posterior Predictive Example



Variational Inference: The Optimisation View

Core idea: replace sampling with optimisation. Find $q_\phi(w)$ that minimises:

$$D_{\text{KL}}(q_\phi(w) \parallel p(w|\mathcal{D})) \equiv \max_{\phi} \mathcal{L}(\phi)$$

Learn the weights via Variational Inference

where

$$\mathcal{L}(\phi) = \underbrace{\mathbb{E}_{w \sim q_\phi} [\log p(\mathcal{D}|w)]}_{\text{expected log-likelihood } \uparrow} - \underbrace{D_{\text{KL}}(q_\phi(w) \parallel p(w))}_{\text{KL regulariser } \downarrow}$$

Mean-field Gaussian variational family

$$q_\phi(w) = \prod_j \mathcal{N}(w_j; \mu_j, \sigma_j^2)$$

Each weight w_j has its own learnable (μ_j, σ_j) .

Number of parameters: $2d$ instead of d .

Reparameterisation: $w_j = \mu_j + \sigma_j \varepsilon_j$, $\varepsilon_j \sim \mathcal{N}(0, 1)$

Analytic KL for Gaussian families

$$D_{\text{KL}}(q_\phi \parallel p) = \frac{1}{2} \sum_j \left[\frac{\mu_j^2 + \sigma_j^2}{\tau^2} - 1 - \log \frac{\sigma_j^2}{\tau^2} \right]$$

This term is differentiable: gradients flow directly from the KL to (μ_j, σ_j) .

Bayes by Backprop (Blundell et al., 2015)

Algorithm

Store mean μ and (softplus-parameterised) std $\sigma = \text{softplus}(\rho)$ for every weight.

Per mini-batch $\mathcal{B}$:

- 1 Sample $\varepsilon \sim \mathcal{N}(0, I)$; compute $w = \mu + \sigma \odot \varepsilon$
- 2 Estimate the stochastic ELBO:

$$\hat{\mathcal{L}} = \frac{N}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log p(y_i | x_i, w) - D_{\text{KL}}(q_\phi(w) \| p(w))$$

- 3 Gradient step on (μ, ρ) via backprop

Bayes by Backprop: Full Training Algorithm

BbB training algorithm (Blundell 2015)

- 1: **Initialise:** μ and ρ ($\sigma = \text{softplus}(\rho) \approx 0.05$)
- 2: **for** each mini-batch $\mathcal{B}$ of size B **do**
- 3: **Sample** noise: $\varepsilon \sim \mathcal{N}(0, I)$
- 4: **Sample** weights: $w = \mu + \text{softplus}(\rho) \odot \varepsilon$
- 5: **Forward pass** with w , compute predictions
- 6: **Compute stochastic ELBO:**

$$\hat{\mathcal{L}} = \frac{N}{B} \sum_{i \in \mathcal{B}} \log p(y_i | x_i, w) - D_{\text{KL}}(q_\phi \| p)$$

- 7: **Backprop** through μ and ρ (reparameterisation)
- 8: **Adam/SGD** update on (μ, ρ)
- 9: **end for**
- 10: **Return:** $q_\phi(w) = \prod_j \mathcal{N}(w_j; \mu_j, \text{softplus}(\rho_j)^2)$

PyTorch BbB linear layer

```
class BayesLinear(nn.Module):
    def __init__(self, in_f, out_f,
                  prior_std=1.0):
        super().__init__()
        self.mu = nn.Parameter(
            torch.zeros(out_f, in_f))
        self.rho = nn.Parameter(
            torch.full((out_f, in_f), -3.))
        self.prior_std = prior_std

    def forward(self, x):
        sigma = F.softplus(self.rho)
        eps = torch.randn_like(sigma)
        w = self.mu + sigma * eps
        # KL for this layer
        kl = 0.5 * (
            (self.mu**2 + sigma**2)
            / self.prior_std**2
            - 1 - 2 * torch.log(sigma)
            + 2 * math.log(self.prior_std)
        ).sum()
        return F.linear(x, w), kl
```


BbB: Inference and Practical Pitfalls

Test-time inference

```
1: for  $s = 1, \dots, S$  do    $S$  # of networks
2:   Sample  $\varepsilon \sim \mathcal{N}(0, I)$ 
3:    $w^{(s)} = \mu + \text{softplus}(\rho) \odot \varepsilon$ 
4:    $\hat{y}^{(s)} = f_{w^{(s)}}(x^*)$ 
5: end for
6:  $\hat{\mu} = \frac{1}{S} \sum_s \hat{y}^{(s)}$ 
7:  $\hat{\sigma}_{\text{ep}}^2 = \frac{1}{S} \sum_s (\hat{y}^{(s)} - \hat{\mu})^2$ 
```

KL cost annealing trick

Early training: the KL term can dominate and prevent the model from fitting data (posterior collapse).

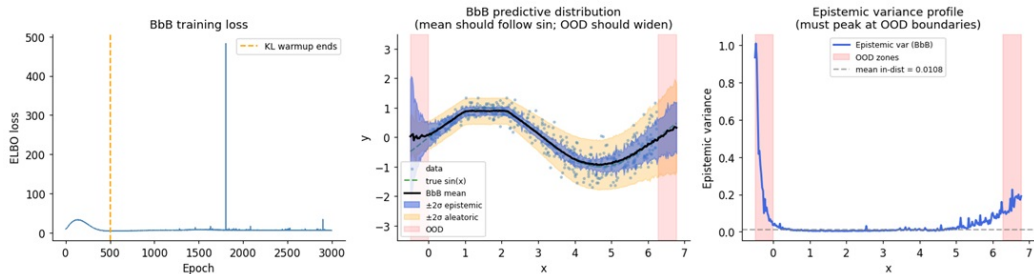
Solution: multiply KL by a coefficient β that increases from 0 to 1 over the first T_{warm} epochs.

Common pitfalls

- 1 **Double parameters:** BbB doubles trainable parameters vs. standard NN — memory cost.
- 2 **High gradient variance:** single-sample ELBO is noisy. Use local reparameterisation: sample pre-activations $a = x^T \mu + x^T (\sigma \odot \varepsilon)$ instead of w .
- 3 **Prior choice:** τ controls posterior regularisation. Too small $\Rightarrow$ weights shrink; too large $\Rightarrow$ prior is uninformative and KL vanishes.
- 4 **Initialisation:** start ρ at -3 so $\sigma \approx 0.05$ (small initial uncertainty).

Variational Inference Example

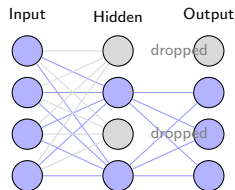
Bayes-by-Backprop: fixed (prior_std=0.3, hidden=128)



Standard Dropout (Srivastava et al., 2014) — Recap

During training: randomly zero each neuron with probability p :

$$\tilde{h}_j = h_j \cdot m_j, \quad m_j \sim \text{Bernoulli}(1 - p)$$
$$m_j = \begin{cases} 1 & \text{with probability } 1 - p \quad (\text{keep neuron}), \\ 0 & \text{with probability } p \quad (\text{drop neuron}). \end{cases}$$



drop some neurons during training

Training:

- Each forward pass uses a different mask $\mathbf{m}^{(s)}$
- Acts as a regulariser (implicit ensemble)

Standard test time:

- Dropout *disabled*
- Weights scaled by $(1 - p)$
- Returns a **single** prediction

MC Dropout: Keep Dropout On at Test Time

Key idea (Gal & Ghahramani, 2016): leave dropout *enabled* during inference and run T stochastic forward passes.

MC Dropout inference

For test input x^* , sample T masks $\mathbf{m}^{(1)}, \dots, \mathbf{m}^{(T)}$:

$$\hat{y}^{(t)} = f_{\hat{w}, \mathbf{m}^{(t)}}(x^*), \quad t = 1, \dots, T$$

- **Predictive mean:** $\hat{\mu}(x^*) = \frac{1}{T} \sum_{t=1}^T \hat{y}^{(t)}$
- **Epistemic variance:** $\hat{\sigma}_{\text{ep}}^2(x^*) = \frac{1}{T} \sum_{t=1}^T (\hat{y}^{(t)} - \hat{\mu})^2$
- **Total variance** (if model also outputs $\hat{\sigma}_{\text{al}}^2$):

$$\hat{\sigma}_{\text{total}}^2 = \hat{\sigma}_{\text{ep}}^2 + \frac{1}{T} \sum_{t=1}^T \hat{\sigma}_{\text{al}}^{2,(t)}$$

Variational Interpretation of MC Dropout

Gal & Ghahramani (2016): dropout training approximately minimises a **variational objective**.

Dropout as VI (key result)

Let $\mathbf{m} \sim \text{Bernoulli}(1 - p)^d$ be the dropout mask. Define the variational distribution:

$$q(w) = q(\mathbf{M}, \boldsymbol{\mu}), \quad W_l = \text{diag}(\mathbf{m}_l) \cdot M_l$$

Then training with dropout $\approx$ minimising:

$$\min_{q(w)} \mathbb{E}_{w \sim q}[-\log p(\mathcal{D})] + D_{\text{KL}}(q(w) \parallel p(w))$$

Practical upshot

Each dropout mask $\mathbf{m}^{(t)}$ is a **sample** from an implicit approximate posterior over weights.

MC averaging over T masks $\approx$ Bayesian model averaging.

Important caveats

- The variational family is **restrictive**
- Uncertainty quality depends on dropout rate p , placement, and T
- May underestimate or misrepresent uncertainty for some problems

MC Dropout: Exact Training Procedure

Training (regression with aleatoric noise)

- 1: Build network with **Dropout(p)** after each hidden layer
- 2: Output head: two scalars $(\hat{\mu}, \log \hat{\sigma}^2)$ per sample
- 3: **for** each mini-batch $\mathcal{B}$ **do**
- 4: Forward pass (**dropout is active by default**)
- 5: Compute heteroscedastic NLL:

$$\mathcal{L} = \frac{1}{B} \sum_{i \in \mathcal{B}} \left[\frac{(y_i - \hat{\mu}_i)^2}{2e^{s_i}} + \frac{s_i}{2} \right]$$

where $s_i = \log \hat{\sigma}_i^2$

- 6: Backprop + Adam/SGD on (point estimate)
- 7: **end for**
- 8: **No changes to training vs. standard NN**

MC Dropout: Full Inference Procedure

Algorithm (regression)

Training: standard dropout + NLL loss
(no changes needed)

Test-time inference:

- 1: **for** $t = 1, \dots, T$ **do**
- 2: sample mask $\mathbf{m}^{(t)} \sim \text{Ber}(1 - p)^d$
- 3: $\hat{y}^{(t)}, \hat{\sigma}^{2,(t)} \leftarrow f_{\hat{\mathbf{w}}, \mathbf{m}^{(t)}}(x^*)$
- 4: **end for**
- 5: $\hat{\mu} \leftarrow \frac{1}{T} \sum_t \hat{y}^{(t)}$
- 6: $\hat{\sigma}_{\text{ep}}^2 \leftarrow \frac{1}{T} \sum_t (\hat{y}^{(t)} - \hat{\mu})^2$
- 7: $\hat{\sigma}_{\text{al}}^2 \leftarrow \frac{1}{T} \sum_t \hat{\sigma}^{2,(t)}$
- 8: $\hat{\sigma}_{\text{total}}^2 \leftarrow \hat{\sigma}_{\text{ep}}^2 + \hat{\sigma}_{\text{al}}^2$

PyTorch one-liner

Enable dropout at test:

```
model.eval()  
for m in model.modules():  
    if isinstance(m, nn.Dropout):  
        m.train()
```

Then loop T forward passes.

Choosing T

$T \in [20, 100]$ is typically sufficient.
Plot variance-of-variance vs T to diagnose convergence.
 $T=50$ is a sensible default.

Why VI (Bayes-by-Backprop) When MC Dropout Exists?

MC Dropout requires no architecture changes — so why would anyone do VI?

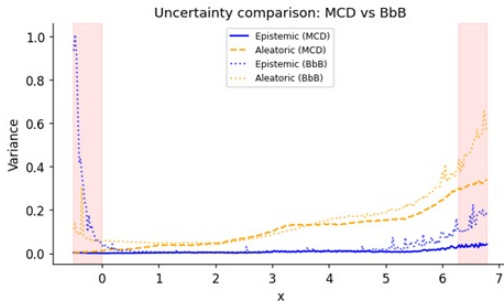
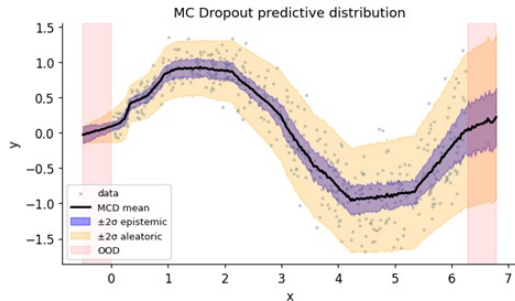
MC Dropout strengths

- **Zero training overhead:** retrofit onto any existing model
- **Same parameter count:** no memory increase
- **No special layers needed**
- **Works for any architecture:** CNNs, RNNs, Transformers
- Inference cost: $T \times$ one forward pass

MC Dropout limitations

- Restricted variational family: Bernoulli masks only
- Dropout rate p is a fixed hyperparameter — may be wrong
- Uncertainty quality depends sensitively on p and placement
- Cannot capture *weight correlations* across layers
- Often **underestimates epistemic uncertainty** on OOD data

MC Dropout Predictive distribution

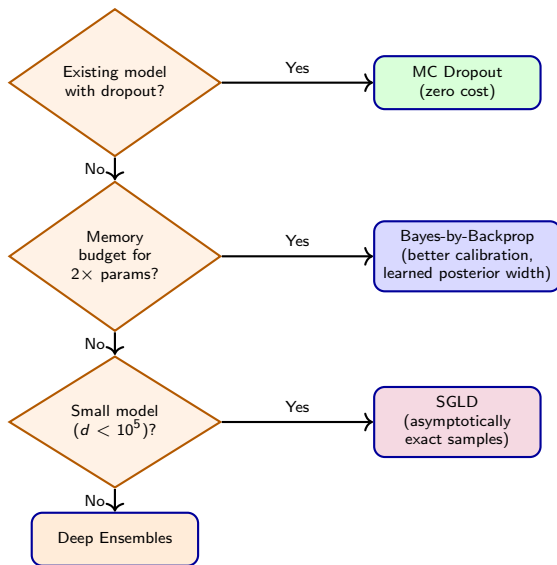


- **BbB epistemic** (blue dotted): large, peaks sharply at both OOD boundaries — *rich posterior*.
- **MCD epistemic** (blue solid): small, barely rises at OOD — *coarser posterior approximation*.
- **Both aleatoric curves** (orange): similar shape, growing rightward — consistent learned noise, as expected since both methods use the same heteroscedastic NLL loss.

Takeaway

MC Dropout trades *posterior richness* for *simplicity*: no extra parameters, no KL term, easy to retrofit to any existing network — but at the cost of less expressive uncertainty estimates, particularly for OOD detection.

MC Dropout vs. BbB vs. SGLD: Decision Guide



Uncertainty Quality: A Comparison

Property	MC Dropout	BbB	SGLD	Ensembles
Training cost	$1\times$	$1.5\times$	$1\times$ (long)	$M\times$
Parameter overhead	None	$2\times$	None (checkpoints)	$M\times$
Inference cost	$T\times$ fwd	$S\times$ fwd	$S\times$ fwd	$M\times$ fwd
Posterior quality	Approximate	Approximate	Asymptotic	Empirical
Calibration	Moderate	Good	Best	Good
OOD detection	Good	Good	Good	Better
Architecture changes	None	Yes (layers)	None	None
Easy to tune	Moderate	Hard	Hard	Easy

When MC Dropout fails

Long-range correlations in weight posterior (deep, narrow networks); very small dropout rates effectively give nearly-deterministic models.

Methods of Approximate Inference for BNNs

MCMC

HMC / NUTS / SG-MCMC

Define prior $p(\mathbf{w})$ and likelihood $p(\mathcal{D} \mid \mathbf{w})$

Run Markov chain targeting posterior $p(\mathbf{w} \mid \mathcal{D})$

Keep posterior samples $\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(S)}$

Predict at x^* using posterior samples

Average predictions + quantify spread (credible intervals)

Sampling-based (high fidelity, expensive)

Variational Inference

Bayes by Backprop

Choose family $q_\phi(\mathbf{w})$ (e.g., factorized Gaussian)

Optimise ELBO w.r.t. ϕ via gradients + reparam.

Obtain approximate posterior $q_{\phi^*}(\mathbf{w})$

Sample $\mathbf{w} \sim q_{\phi^*}(\mathbf{w})$ and predict on x^*

MC average + uncertainty from $q_{\phi^*}(\mathbf{w})$ samples

Optimisation-based (scalable, can be biased)

MC Dropout

Dropout as approx. Bayesian inference

Train network with dropout (+ weight decay)

At test time keep dropout ON

Run T stochastic forward passes with different dropout masks

Collect $\hat{y}_1, \dots, \hat{y}_T$ for x^*

Estimate predictive mean and variance from samples

Practical stochastic approx. (easy to retrofit to NNs)

Target quantity: predictive distribution $p(y^* \mid x^*, \mathcal{D})$

Diagnosing Approximate Inference: Did It Work?

Approximate inference can *look* plausible while being systematically wrong.

Five mandatory sanity checks

- 1 **Epistemic uncertainty shrinks with data:** Hold out x^* ; retrain with more data nearby; verify $\hat{\sigma}_{\text{ep}}^2(x^*)$ decreases.
- 2 **Uncertainty grows away from data:** Evaluate on OOD inputs (e.g. unseen arrhythmia class); epistemic uncertainty should be higher than in-distribution.
- 3 **Posterior samples are diverse:** Plot T stochastic forward passes; they should differ visibly (if they collapse to one line, the posterior has collapsed).
- 4 **Prior sensitivity:** Try $\tau \in \{0.1, 1.0, 10.0\}$; results should be qualitatively robust.
- 5 **NLL on held-out set:** Better UQ method $\equiv$ lower predictive NLL (a proper scoring rule; accuracy alone is not sufficient).

Implementation take-aways

- 1 **Training:** use heteroscedastic NLL loss with two output heads ($\hat{\mu}, \log \hat{\sigma}^2$) to learn both epistemic (via MC dropout/BbB) and aleatoric (via learned variance) uncertainty.
- 2 **SGLD:** inject Gaussian noise proportional to $\sqrt{\epsilon_t}$ into SGD updates; discard burn-in; asymptotically exact but expensive.
- 3 **Bayes-by-Backprop:** store (μ, ρ) per weight; reparameterise; maximise stochastic ELBO with analytic KL; doubles parameter count.
- 4 **MC Dropout:** cheapest retrofit; keep dropout on at test time with `model.eval()+enable_dropout()`; $T = 50$ passes; freeze BatchNorm.
- 5 **Choice rule:** MCD first; BbB if you can redesign; SGLD for benchmarks; Ensembles for safety-critical.
- 6 **Biomedical applications:** ECG OOD arrhythmia detection, PPG SpO₂ noise decomposition, EEG active annotation, EMG sensor drift alerting.

Key References

- **Gal & Ghahramani (2016):** “Dropout as a Bayesian Approximation” — MC Dropout theory.
- **Blundell et al. (2015):** “Weight Uncertainty in Neural Networks” — Bayes-by-Backprop.
- **Kendall & Gal (2017):** “What Uncertainties Do We Need?” — epistemic/aleatoric decomposition.
- **Welling & Teh (2011):** “Bayesian Learning via Stochastic Gradient Langevin Dynamics” — SGLD.
- **Lakshminarayanan et al. (2017):** “Simple and Scalable Predictive Uncertainty using Deep Ensembles.”
- **Zhang et al. (2020):** “Cyclical Stochastic Gradient MCMC for Bayesian Deep Learning.”
- **Ovadia et al. (2019):** “Can You Trust Your Model’s Uncertainty?” — benchmarks under dataset shift.
- **Gawlikowski et al. (2023):** “A Survey of Uncertainty in Deep Neural Networks.”

Questions?